# اخبرني

# Akbrny Platform

## Final Client Technical Report

## Laravel 5.8 → Laravel 13.24.0 Upgrade

---

Anonymous messaging & polls platform modernization

Report date: 12 August 2026

Branch: production-readiness

Verified Laravel version: 13.24.0

| Metric | Verified Result |
|---|---|
| Laravel | 5.8 → 13.24.0 |
| PHP compatibility | ^8.3 (test server PHP 8.3.33) |
| Tests | 51 / 51 passing (211 assertions) |
| API contract tests | 33 / 33 |
| FCM tests | 9 / 9 |
| API v1 tests | 4 / 4 |
| Legacy AJAX endpoints preserved | 14 / 14 |
| AJAX controllers / Services | 7 / 7 (+ FCM & Storage services) |
| Production database | NOT modified |

This report documents only work that was actually implemented and verified. Items that require server access or production deployment are labeled clearly.

Status labels used throughout: IMPLEMENTED · VERIFIED · REQUIRES SERVER VERIFICATION · PENDING PRODUCTION DEPLOYMENT

# Table of Contents

# 1. Executive Summary

The Akbrny (اخبرني) platform was upgraded from Laravel 5.8 to Laravel 13.24.0 with PHP 8.3 compatibility, a full AJAX architecture refactor, Eloquent query modernization, API v1, Firebase FCM HTTP v1, Laravel Storage for profile images, caching for static pages, queued notifications, and rate limiting.

All 14 legacy AJAX endpoints retain their original URLs, HTTP methods, and route names. The production database was not modified. Application migrations were verified on a separate test database (`akbrny2026_akbrny_test`).

## Key verified outcomes

| Area | Result | Status |
|---|---|---|
| Framework | Laravel 13.24.0 | VERIFIED |
| PHP | ^8.3 / test server 8.3.33 | VERIFIED |
| Automated tests | 51 tests, 211 assertions | VERIFIED |
| AJAX compatibility | 14/14 endpoints preserved | VERIFIED |
| API v1 | 5 endpoints | IMPLEMENTED |
| Production DB | Not modified | VERIFIED |

Screenshot note: Directory `docs/screenshots/final/` was not initially present. Available verified screenshots from `docs/screenshots/` were linked into `final/` for this report. Topics without a real image are marked “Screenshot unavailable” — no images were fabricated.

# 2. Laravel 5.8 → Laravel 13.24.0 Upgrade

**VERIFIED**

The application framework was upgraded from Laravel 5.8 to Laravel 13.24.0. This includes modern bootstrap structure (`bootstrap/app.php`), updated middleware configuration, and Composer dependency resolution aligned with Laravel 13.

```
php artisan --version → Laravel Framework 13.24.0
```

Major upgrade commit: 469d783 — feat: upgrade project to Laravel 13 and PHP 8.4 (later aligned to PHP 8.3 in 4e4a844).

# 3. PHP 8.3 Compatibility

**VERIFIED**

Composer requires PHP `^8.3` with platform pin `8.3.30` to keep dependency resolution on PHP 8.3-compatible packages. The test server runs PHP 8.3.33.

| Item | Value | Status |
|---|---|---|
| composer.json php | ^8.3 | IMPLEMENTED |
| platform.php | 8.3.30 | IMPLEMENTED |
| Test server PHP | 8.3.33 | VERIFIED |

| Item | Value | Status |
|---|---|---|
| composer check-platform-reqs | Pass | VERIFIED |

# 4. Composer & Symfony Dependency Alignment

**VERIFIED**

After the initial Laravel 13 upgrade targeted PHP 8.4, dependencies were re-aligned for PHP 8.3. Symfony components were resolved to the 7.4.x line (Symfony 8.x requires PHP ≥ 8.4.1). Laravel itself remained at 13.24.0 — Laravel 13 officially supports PHP 8.3.

Commit: 4e4a844 — fix(deps): align Laravel 13 dependencies with PHP 8.3

# 5. AJAX Architecture Refactor

**IMPLEMENTED**

The monolithic `AjaxController` (~708 lines) was split into focused controllers under `app/Http/Controllers/Ajax/`, with business logic moved into service classes. Routes remain grouped under `/ajax` with identical URLs, methods, and names.

## Project structure after refactor

```
Project structure AFTER refactor
app/Http/Controllers/
├── Ajax/
│   ├── EmailCheckController.php
│   ├── MessageController.php
│   ├── QuestionController.php
│   ├── UserAccountController.php
│   ├── ProfileVoteController.php
│   ├── NotificationController.php
│   └── SearchController.php
├── Api/V1/
│   ├── SearchController.php
│   ├── MessageController.php
│   ├── PollController.php
│   ├── NotificationController.php
│   └── UserController.php
├── User/
│   ├── ProfileController.php
│   └── UsersController.php
└── Auth/ ...

app/Services/
├── EmailAvailabilityService.php
├── MessageService.php
├── QuestionService.php
├── UserAccountService.php
├── PollVoteService.php
├── SiteSearchService.php
├── NotificationTokenService.php
└── FirebaseCloudMessaging.php
```

Figure: Application structure — Ajax controllers, Api/V1, and Services.

# 6. Controllers Organization

**IMPLEMENTED**

| Controller | Responsibility |
|---|---|
| EmailCheckController | Email availability check |

| Controller | Responsibility |
|---|---|
| MessageController | Reply / edit / delete messages |
| QuestionController | Add poll/question posts |
| UserAccountController | Profile, password, image, settings, social |
| ProfileVoteController | Poll voting |
| NotificationController | FCM token registration |
| SearchController | Site search |

Ajax\MessageController

```php
<?php

namespace App\Http\Controllers\Ajax;

use App\Http\Controllers\Controller;
use App\Services\MessageService;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Validator;

class MessageController extends Controller
{
    public function __construct(
        private readonly MessageService $messages,
    ) {}

    public function replyMessage(Request $request)
    {
        $error = Validator::make($request->all(), [
            'reply' => 'required',
            'message_reply_id' => 'required',
        ]);

        if ($error->fails()) {
            return response()->json(['errors' => $error->errors()->all()]);
        }

        return response()->json($this->messages->reply(
            auth()->user()->id,
            (int) $request->get('message_reply_id'),
            $request->get('reply'),
        ));
    }

    public function deleteReplyMessage(Request $request)
    {
        return response()->json($this->messages->deleteReply(
            auth()->user()->id,
            (int) $request->get('reply_id'),
        ));
    }

    public function editMessage(Request $request)
    {
        return response()->json($this->messages->toggleVisibility(
            auth()->user()->id,
            (int) $request->get('post_id'),
            (int) $request->get('type'),
            (int) $request->get('is_question'),
        ));
    }

    public function deleteMessage(Request $request)
    {
        return response()->json($this->messages->deletePost(
            auth()->user()->id,
```

Figure: Ajax MessageController after split — thin controller, service delegation.

# 7. Service Layer

**IMPLEMENTED**

Domain services encapsulate Eloquent queries and business rules shared by AJAX and API controllers.

| Service | Role |
|---|---|
| EmailAvailabilityService | Email uniqueness checks |
| MessageService | Message reply/edit/delete |
| QuestionService | Poll/question creation |
| UserAccountService | Profile & account updates |
| PollVoteService | Voting logic |
| SiteSearchService | Search queries |
| NotificationTokenService | Device token persistence |

| Service | Role |
| --- | --- |
| FirebaseCloudMessaging | FCM HTTP v1 send |
| ProfileImageStorage | Profile image Storage disk ops |

MessageService — Eloquent + exists()

```php
<?php

namespace App\Services;

use App\Answer;
use App\Post;

class MessageService
{
    public function reply(int $userId, int $messageReplyId, string $reply): array
    {
        if (! Post::where('user_id', $userId)->where('id', $messageReplyId)->exists()) {
            return ['errors' => trans('main.error_somethings')];
        }

        if (Answer::where('user_id', $userId)->where('post_id', $messageReplyId)->exists()) {
            return ['errors' => trans('main.message_already_reply')];
        }

        Answer::create([
            'post_id' => $messageReplyId,
            'user_id' => $userId,
            'body' => $reply,
        ]);

        return ['success' => trans('main.message_done_reply')];
    }

    public function deleteReply(int $userId, int $replyId): array
    {
        if (! Answer::where('user_id', $userId)->where('id', $replyId)->exists()) {
            return ['errors' => trans('main.error_somethings')];
        }

        $deleted = Answer::where('user_id', $userId)->where('id', $replyId)->delete();

        if ($deleted === 1) {
            return ['success' => trans('main.message_done_reply')];
        }

        return ['errors' => trans('main.message_already_reply')];
    }

    public function toggleVisibility(int $userId, int $postId, int $type, int $isQuestion): array
    {
        if (! Post::where('user_id', $userId)->where('id', $postId)->exists()) {
            return ['errors' => trans('main.error_somethings')];
        }

        $isPublic = $type === 1 ? 0 : 1;

        $updated = Post::where('id', $postId)
            ->where('user_id', $userId)
            ->update($isQuestion === 1 ? ['is_active' => $isPublic] : ['is_public' => $isPublic]);

```

Figure: MessageService — Eloquent-based business logic.

# 8. Database Query Optimization

**IMPLEMENTED**

Query execution was optimized at the application level. Raw Query Builder usage was reduced in favor of Eloquent models, `exists()` instead of unnecessary `count()` checks, and targeted selects where appropriate.

Production response-time improvement requires production benchmarking. No percentage speed claim is made in this report.

# 9. Eloquent Conversion

**IMPLEMENTED**

Approximately 25 Query Builder usages were converted to Eloquent across services and providers (for example unread message counts, vote lookups, answer creation, and email checks).

Commit: `be6f62e` — refactor: split AJAX monolith, add services, API v1, and Eloquent queries

# 10. N+1 Query Fix

**IMPLEMENTED**

Profile poll pages previously risked N+1 queries when checking viewer votes per poll. Votes are now batch-loaded once and passed to the view as `$viewerAnswersByPostId`.

ProfileController — batch viewer answers (N+1 fix)

```
        $posts = $allPosts->where('type', 0);
        $polls = $allPosts->where('type', 1);
        $posts_polls = $allPosts->where('is_public', 1)->sortByDesc('id')->values();

        $viewerAnswersByPostId = collect();
        if (auth()->check()) {
            $pollIds = $allPosts->where('type', 1)->pluck('id');
            if ($pollIds->isNotEmpty()) {
                $viewerAnswersByPostId = Answer::query()
                    ->where('user_id', auth()->id())
                    ->whereIn('post_id', $pollIds)
                    ->get()
                    ->keyBy('post_id');
            }
        }

        if(!session()->has($username)){
            session([$username => 'true']);
            $user->visitors=$user->visitors+1;
            $user->save();
```

Figure: ProfileController N+1 fix — batch Answer lookup keyed by post_id.

# 11. Database Index Optimization

**IMPLEMENTED**

Additive performance indexes were defined in migration `2026_08_08_040705_add_performance_indexes_to_posts_and_answers_tables`:

| Table | Index | Supports |
|---|---|---|
| posts | user_id, is_read, type | Unread counts / dashboard updates |
| answers | user_id, post_id | Vote/reply duplicate checks |

Applied successfully on the test database. Production application of this migration: PENDING PRODUCTION DEPLOYMENT / REQUIRES SERVER VERIFICATION.

Performance indexes migration

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Additive performance indexes (Optimization #5).
     *
     * posts (user_id, is_read, type):
     *   - AppServiceProvider view composer unread COUNT
     *   - UsersController dashboard mark-as-read UPDATE
     *
     * answers (user_id, post_id):
     *   - AjaxController message reply duplicate check (D2)
     *   - AjaxController profileSendVote duplicate vote check (D12)
     *   - Authenticated profile poll viewer vote lookup
     */
    public function up(): void
    {
        Schema::table('posts', function (Blueprint $table) {
            $table->index(['user_id', 'is_read', 'type'], 'posts_user_id_is_read_type_index');
        });

        Schema::table('answers', function (Blueprint $table) {
            $table->index(['user_id', 'post_id'], 'answers_user_id_post_id_index');
        });
    }

    public function down(): void
    {
        Schema::table('posts', function (Blueprint $table) {
            $table->dropIndex('posts_user_id_is_read_type_index');
        });

        Schema::table('answers', function (Blueprint $table) {
            $table->dropIndex('answers_user_id_post_id_index');
        });
    }
};
```

Figure: Performance index migration source.

# 12. API v1

**IMPLEMENTED**

A versioned JSON API was added under `/api/v1`, reusing the same service layer as AJAX endpoints.

| Method | Endpoint | Auth |
|---|---|---|
| GET | /api/v1/search | Public |
| POST | /api/v1/messages/reply | auth |
| POST | /api/v1/polls/vote | auth |
| POST | /api/v1/notifications/token | auth |
| PUT | /api/v1/users/profile | auth |

routes/api.php — v1 JSON API

```php
<?php

use App\Http\Controllers\Api\V1\MessageController as ApiMessageController;
use App\Http\Controllers\Api\V1\NotificationController as ApiNotificationController;
use App\Http\Controllers\Api\V1\PollController as ApiPollController;
use App\Http\Controllers\Api\V1\SearchController as ApiSearchController;
use App\Http\Controllers\Api\V1\UserController as ApiUserController;
use Illuminate\Support\Facades\Route;

/*
|--------------------------------------------------------------------------
| API Routes — v1 (JSON, for mobile / external clients)
|--------------------------------------------------------------------------
|
| Legacy frontend continues to use /ajax/* routes in routes/web.php.
| These endpoints share the same Service layer with zero contract change
| on existing AJAX URLs.
|
*/

Route::prefix('v1')->group(function () {
    Route::get('/search', [ApiSearchController::class, 'index']);

    Route::middleware('auth')->group(function () {
        Route::post('/messages/reply', [ApiMessageController::class, 'reply']);
        Route::post('/polls/vote', [ApiPollController::class, 'vote']);
        Route::post('/notifications/token', [ApiNotificationController::class, 'store']);
        Route::put('/users/profile', [ApiUserController::class, 'updateProfile']);
    });
});

// Legacy Laravel stub — preserved for backward compatibility.
Route::middleware('auth:api')->get('/user', function (\Illuminate\Http\Request $request) {
    return $request->user();
});
```

Figure: API v1 routes definition.

Api\V1\SearchController

```php
<?php

namespace App\Http\Controllers\Api\V1;

use App\Http\Controllers\Controller;
use App\Services\SiteSearchService;
use Illuminate\Http\Request;

class SearchController extends Controller
{
    public function __construct(
        private readonly SiteSearchService $search,
    ) {}

    /**
     * GET /api/v1/search?query=
     */
    public function index(Request $request)
    {
        return response()->json($this->search->searchJson((string) $request->query('query', '')));
    }
}
```

Figure: API SearchController example.

# 13. Legacy AJAX Compatibility

**VERIFIED**

All 14 legacy AJAX endpoints were preserved with unchanged URLs, HTTP methods, and route names. 33/33 API contract tests pass against these contracts.

routes/web.php — grouped AJAX routes

```php
<?php

use App\Http\Controllers\Ajax\EmailCheckController;
use App\Http\Controllers\Ajax\MessageController as AjaxMessageController;
use App\Http\Controllers\Ajax\NotificationController as AjaxNotificationController;
use App\Http\Controllers\Ajax\ProfileVoteController;
use App\Http\Controllers\Ajax\QuestionController;
use App\Http\Controllers\Ajax\SearchController as AjaxSearchController;
use App\Http\Controllers\Ajax\UserAccountController;
use App\Http\Controllers\HomeController;
use App\Http\Controllers\User\ProfileController;
use App\Http\Controllers\User\UsersController;
use Illuminate\Support\Facades\Auth;
use Illuminate\Support\Facades\Route;

Auth::routes();

Route::get('/', [HomeController::class, 'index'])->name('home');

Route::prefix('user')->middleware('auth')->group(function () {
    Route::get('', [UsersController::class, 'index'])->name('user.index');
    Route::get('settings', [UsersController::class, 'settings'])->name('user.settings');
    Route::get('notification', [UsersController::class, 'notification'])->name('user.notification');
});

Route::get('{username?}', [ProfileController::class, 'getUser'])->name('user.getUser');
Route::post('{username?}', [ProfileController::class, 'senMessageToUser'])->name('profile.senMessageToUser');

Route::prefix('ajax')->group(function () {
    Route::post('email/check', [EmailCheckController::class, 'checkEmail'])->name('email_available.check');

    Route::post('message/reply', [AjaxMessageController::class, 'replyMessage'])->name('ajax.reply_message');
    Route::post('message/delete_reply', [AjaxMessageController::class, 'deleteReplyMessage'])->name('ajax.dele
    Route::post('message/edit', [AjaxMessageController::class, 'editMessage'])->name('ajax.edit_message');
    Route::post('message/delete', [AjaxMessageController::class, 'deleteMessage'])->name('ajax.delete_message'

    Route::post('question/add', [QuestionController::class, 'addQuestion'])->name('ajax.question_add');

    Route::post('user/edit_info', [UserAccountController::class, 'userEditInfo'])->name('ajax.userEditInfo');
    Route::post('user/changePassword', [UserAccountController::class, 'userChangePassword'])->name('ajax.userC
    Route::post('user/ChangeImage', [UserAccountController::class, 'userChangeImage'])->name('ajax.userChangeI
    Route::post('user/EditSettings', [UserAccountController::class, 'userEditSettings'])->name('ajax.userEditS
    Route::post('user/userEditSocial', [UserAccountController::class, 'userEditSocial'])->name('ajax.userEditS
    Route::post('user/saveNotificationToken', [AjaxNotificationController::class, 'saveNotificationToken'])->n

    Route::post('profile/profileSendVote', [ProfileVoteController::class, 'profileSendVote'])->name('ajax.prof

    Route::get('site/search', [AjaxSearchController::class, 'siteSearch'])->name('ajax.siteSearch');
});

Route::get('pages/contact', fn () => view('pages.contact'))->name('pages.contact');
Route::get('pages/privacy-policy', fn () => view('pages.privacy-policy'))->name('pages.privacy-policy');
Route::get('pages/terms', fn () => view('pages.terms'))->name('pages.terms');
```

Figure: Web + AJAX routes after refactor (same contracts).

# 14. Laravel Cache

**IMPLEMENTED**

Static informational pages (contact, privacy policy, terms) are served through `PageController` with a 24-hour Cache::remember TTL. User-specific data (messages, votes, auth state, notifications) is intentionally not cached.

Recommended production driver: `CACHE_STORE=file` (Redis not required). Production driver configuration: REQUIRES SERVER VERIFICATION.

Screenshot unavailable for cache implementation UI/code capture under docs/screenshots/final/. Implementation is verified in source (`PageController`) and `StaticPageCacheTest`.

# 15. Laravel Queues / Jobs

**IMPLEMENTED**

FCM notifications are dispatched via `SendFcmNotificationJob` after a message is saved. The HTTP response returns immediately; the worker sends the push.

| Item | Detail | Status |
|---|---|---|
| Job class | app/Jobs/SendFcmNotificationJob.php | IMPLEMENTED |
| Retries / backoff | 3 tries; 10s, 30s, 60s | IMPLEMENTED |
| Queue driver (example) | database (no Redis required) | IMPLEMENTED |
| jobs table migration | Exists in repository | IMPLEMENTED |
| failed_jobs migration | Exists in repository | IMPLEMENTED |
| Queue worker on server | Supervisor recommended | REQUIRES SERVER VERIFICATION |
| Tests | QUEUE_CONNECTION=sync | VERIFIED |

Screenshot unavailable for Jobs directory under docs/screenshots/final/. Source and tests verify the job dispatch path.

# 16. Firebase FCM HTTP v1

**IMPLEMENTED**

Push notifications use Firebase Cloud Messaging HTTP v1 with OAuth2 service-account JWT signing. Credentials are loaded from environment variables only (never committed).

| Config key (env) | Purpose |
|---|---|
| FIREBASE_PROJECT_ID | Firebase project |
| FIREBASE_CLIENT_EMAIL | Service account email |
| FIREBASE_PRIVATE_KEY | Service account private key |
| FIREBASE_REQUEST_TIMEOUT | HTTP timeout (seconds) |

Automated tests use `Http::fake` — no real production notifications are sent during CI/local tests. FCM suite: 9/9 passing.

Screenshot unavailable for FCM service source under docs/screenshots/final/. Verified via FcmNotificationTest and FirebaseCloudMessaging service.

# 17. Laravel Storage & Image Handling

IMPLEMENTED

Profile image upload/delete uses Laravel Storage with a dedicated `profile_images` disk rooted at `public/images/profile/`. Legacy public URLs (`/images/profile/{filename}`) are preserved so existing images remain accessible.

Avatar fallback remains `img/avatar2.png`. Commit: `c71c085` — refactor(storage): modernize file and image handling.

Screenshot unavailable for Storage configuration under docs/screenshots/final/. Verified via ProfileImageStorageTest.

# 18. Rate Limiting

IMPLEMENTED

Laravel RateLimiter definitions protect public and sensitive endpoints without blocking normal use.

| Limiter | Limit | Applied to |
|---|---|---|
| login | 10 / min / IP | LoginController |
| register | 5 / min / IP | RegisterController |
| password-reset | 5 / min / IP | Forgot/Reset password |
| messages | 20 / min / user\|IP | Message AJAX + profile send |
| votes | 30 / min / user\|IP | Poll vote AJAX |
| search | 60 / min / IP | Site search |
| notification-token | 10 / min / user\|IP | Token save |
| api | 120 / min / user\|IP | API middleware |

Screenshot unavailable for rate-limiter source under docs/screenshots/final/. Verified via RateLimitTest (429 after limit).

# 19. Frontend Build Verification

VERIFIED

The project uses Laravel Mix 4 (not Vite). Commands `npm install` and `npm run production` completed successfully locally, producing `public/js/app.js` and `public/css/app.css`.

Important: Blade layouts continue to load UI assets from `public/style/`. Mix outputs are not referenced via `mix()` and are gitignored as build artifacts. Primary production UI assets remain the existing Bootstrap/style stack.

# 20. Test Server Deployment

VERIFIED

The application was deployed successfully to a test environment running PHP 8.3.33 with a database separate from production. Laravel migrations were applied on the test database only.

Screenshot unavailable for test-server console/UI under docs/screenshots/final/. Deployment success is recorded from verified project status at report time.

# 21. Database / Test Database

**VERIFIED**

| Item | Detail | Status |
|---|---|---|
| Test database | akbrny2026_akbrny_test | VERIFIED |
| Migrations on test DB | Applied successfully | VERIFIED |
| Production database | NOT modified | VERIFIED |
| Destructive commands | migrate:fresh / db:wipe not used | VERIFIED |
| MySQL 8 on production host | Confirm with SELECT VERSION() | REQUIRES SERVER VERIFICATION |

# 22. Testing Results

**VERIFIED**

Full suite result (local verification for this report): OK (51 tests, 211 assertions).

| Suite | Count | Result |
|---|---|---|
| Total | 51 | PASS |
| API contract | 33 / 33 | PASS |
| FCM (Http::fake) | 9 / 9 | PASS |
| API v1 feature | 4 / 4 | PASS |
| Rate limiting | 2 | PASS |
| Static page cache | 1 | PASS |
| Profile Storage | 2 | PASS |

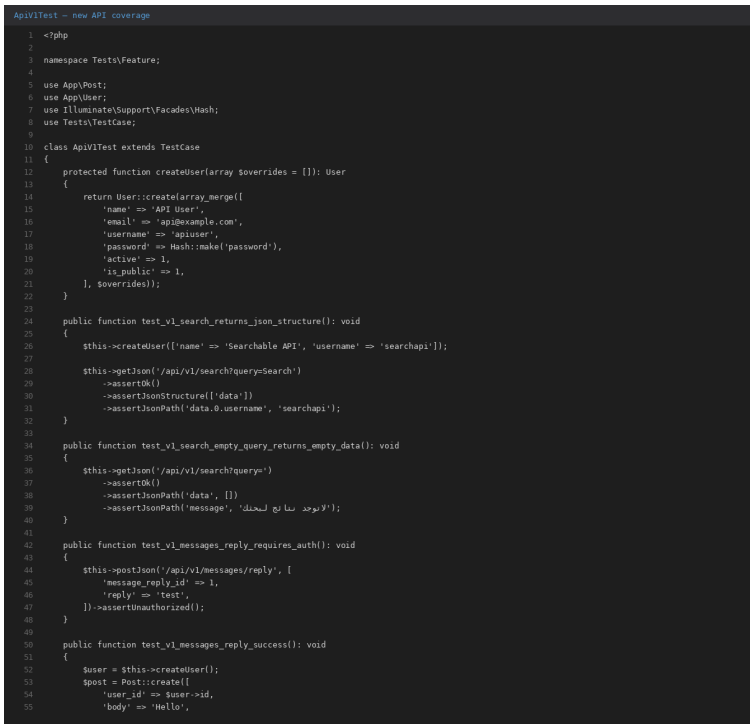


Figure: API v1 / related automated test evidence screenshot.

# 23. Security Improvements

**IMPLEMENTED**

| Control | Status | Notes |
|---|---|---|
| CSRF on web/AJAX | IMPLEMENTED | Laravel web middleware |
| Password hashing | IMPLEMENTED | bcrypt |
| Rate limiting | IMPLEMENTED | Auth, messages, votes, search, API |
| FCM credentials | IMPLEMENTED | Environment variables only |
| APP_DEBUG=false (prod) | PENDING PRODUCTION DEPLOYMENT | Must be set on server |
| Guest AJAX null handling | PRE-EXISTING | Documented; not silently changed |
| Search HTML escaping | PRE-EXISTING | Documented future recommendation |

This PDF deliberately excludes .env contents, passwords, database credentials, API keys, Firebase private keys, tokens, and private user data.

# 24. Performance Improvements

**IMPLEMENTED**

Query execution was optimized at the application level through Eloquent conversion, an N+1 fix on profile polls, additive database indexes, and static page caching.

Production response-time improvement requires production benchmarking. This report does not claim a percentage performance gain.

| Optimization | Status |
|---|---|
| Eloquent / exists() conversions (~25) | IMPLEMENTED |
| N+1 profile poll votes | IMPLEMENTED |
| Composite indexes (posts, answers) | IMPLEMENTED (applied on test DB) |
| Static page cache (24h) | IMPLEMENTED |
| Async FCM via queue job | IMPLEMENTED |
| OPcache / PHP-FPM tuning | REQUIRES SERVER VERIFICATION |

# 25. Git History / Major Commits

**VERIFIED**

| Commit | Summary |
|---|---|
| 469d783 | Upgrade project to Laravel 13 |
| 4e4a844 | Align dependencies with PHP 8.3 |
| be6f62e | Split AJAX, services, API v1, Eloquent |
| c71c085 | Modernize profile image Storage |
| 062164d | Define application rate limiters |
| 108bdbc | Cache static pages + route throttles |
| c9683bd | Queue FCM notifications via Jobs |
| 73dac5d | Verify Laravel Mix production build |
| 16071c7 | Production readiness documentation |

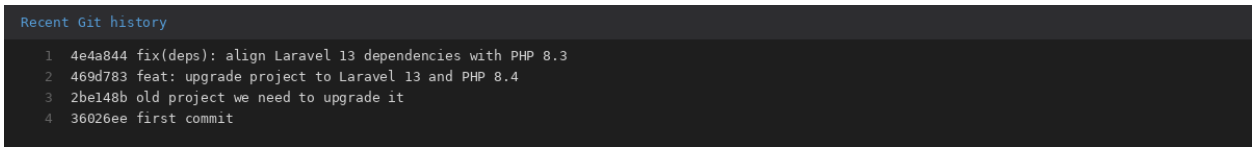


Figure: Git history evidence for architecture refactor commit(s).

Branch `production-readiness` contains the production-readiness commits listed above. No automatic push to production was performed as part of this work.

# 26. Before vs After

| Area | Before (Laravel 5.8 era) | After (Laravel 13) |
|---|---|---|
| Framework | Laravel 5.8 | Laravel 13.24.0 |
| PHP | Legacy / mixed | PHP ^8.3 (test: 8.3.33) |
| AJAX | Single AjaxController monolith | 7 focused Ajax controllers |
| Business logic | Inside controllers | Service layer |
| API | No versioned API v1 | 5 /api/v1 endpoints |

| Area | Before (Laravel 5.8 era) | After (Laravel 13) |
|---|---|---|
| Queries | Heavy Query Builder | ~25 Eloquent conversions |
| N+1 (profile polls) | Per-poll vote queries | Batch load |
| Indexes | Missing composites | Additive migration prepared |
| FCM | Legacy / blocking sync path | HTTP v1 + queued Job |
| Images | Direct public filesystem writes | Storage disk (legacy URLs kept) |
| Cache | None for static pages | 24h Cache::remember |
| Rate limits | Not configured | Named RateLimiters |
| Tests | Limited / older baseline | 51 tests / 211 assertions |

# 27. Production Readiness Status

| Component | Status |
|---|---|
| Laravel 13 + PHP 8.3 application code | VERIFIED |
| Automated test suite | VERIFIED |
| AJAX / API contracts preserved | VERIFIED |
| Cache / Queues / Storage / Rate limiting code | IMPLEMENTED |
| FCM HTTP v1 code + tests | VERIFIED |
| Frontend Mix build (local) | VERIFIED |
| Test DB migrations | VERIFIED |
| Production .env (APP_DEBUG=false, Firebase, DB) | PENDING PRODUCTION DEPLOYMENT |
| Production migrations (jobs, indexes, failed_jobs) | PENDING PRODUCTION DEPLOYMENT |
| Queue worker (Supervisor) | REQUIRES SERVER VERIFICATION |
| Nginx / PHP-FPM / OPcache / SSL | REQUIRES SERVER VERIFICATION |
| MySQL version on production | REQUIRES SERVER VERIFICATION |

# 28. Remaining Server-Side Checks

The following items cannot be completed from the application repository alone:

- Confirm production PHP is 8.3.x with required extensions.
- Confirm MySQL version (SELECT VERSION();) — MySQL 8 recommended.
- Set APP_ENV=production and APP_DEBUG=false.
- Configure Firebase env vars without committing secrets.
- Backup production DB, then run pending additive migrations only.
- Start queue worker for database driver (SendFcmNotificationJob).

• Ensure public/images/profile is writable by PHP-FPM user.

• Enable OPcache in PHP-FPM and tune pool size.

• Verify Nginx document root points to /public and SSL is valid.

• Run production smoke tests (login, message, vote, search, image upload).

• Benchmark production response times before claiming latency improvement.

## 29. Final Summary

The Akbrny platform has been successfully modernized from Laravel 5.8 to Laravel 13.24.0 with PHP 8.3 compatibility, a maintainable AJAX/service architecture, API v1, query optimizations, FCM HTTP v1, Storage-based image handling, caching, queues, and rate limiting.

Application-level work is complete and covered by automated tests (51/51). Production cutover still requires controlled server configuration, additive migrations, queue workers, and production benchmarking.

| Final checklist | Status |
|---|---|
| Laravel 13.24.0 | VERIFIED |
| PHP 8.3 compatibility | VERIFIED |
| 51 tests / 211 assertions | VERIFIED |
| 14/14 AJAX endpoints preserved | VERIFIED |
| Production DB unmodified | VERIFIED |
| Production deployment | PENDING PRODUCTION DEPLOYMENT |
| Server hardening (Nginx/FPM/OPcache) | REQUIRES SERVER VERIFICATION |

— End of report —